



Parallel Algorithms and Parallel Programming – Introduction

058165 - PARALLEL COMPUTING

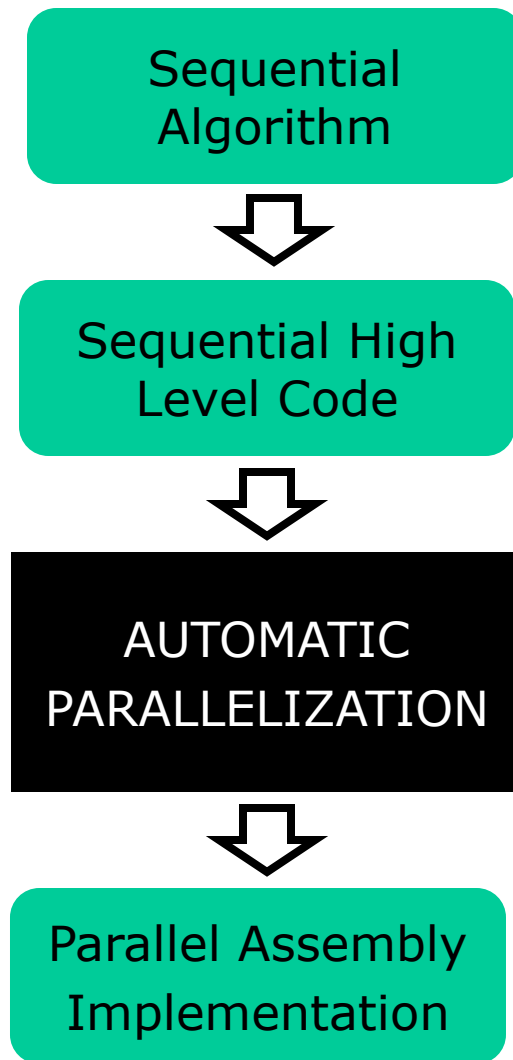
Fabrizio Ferrandi

Politecnico di Milano

Dipartimento di Elettronica, Informazione e Bioingegneria

fabrizio.ferrandi@polimi.it

- ❑ Automatic Parallelization vs. Parallelization by hand
- ❑ Types of parallelism
- ❑ Examples of parallel programming infrastructures



- ❑ Write sequential algorithm
- ❑ Implement with sequential code
- ❑ Leave all the parallelization work to automatic tools

```
void and_or(int SIZE, int * and, int * or, int * b,  
int * c){  
    int i = 0;  
    for(i=0; i<SIZE; i++)  
        and[i] = b[i] & c[i];  
    for(i=0; i<SIZE; i++)  
        or[i] = b[i] | c[i];  
}
```

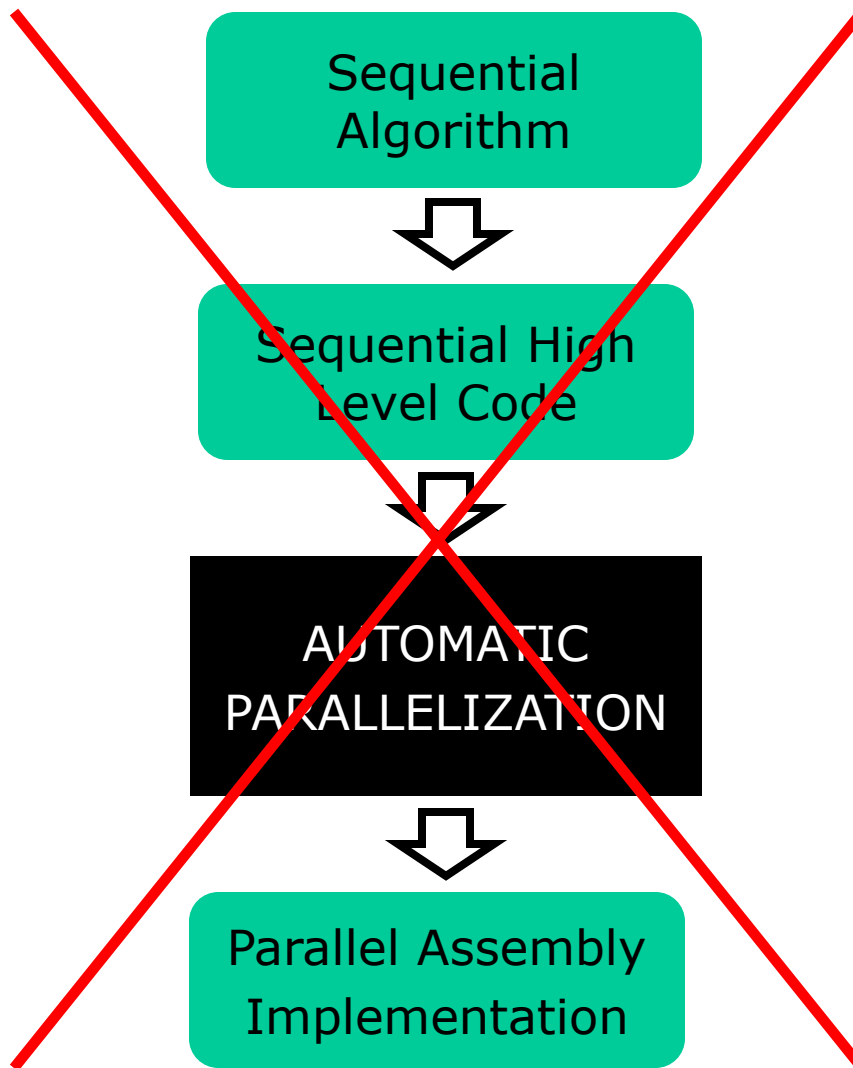
```
void and_or(int SIZE, int * and, int * or, int * b,
int * c){
    int i = 0;
    for(i=0; i<SIZE; i++)
        and[i] = b[i] & c[i];
    for(i=0; i<SIZE; i++)
        or[i] = b[i] | c[i];
}
```

- ❑ This code fragment can not be automatically parallelized as is:

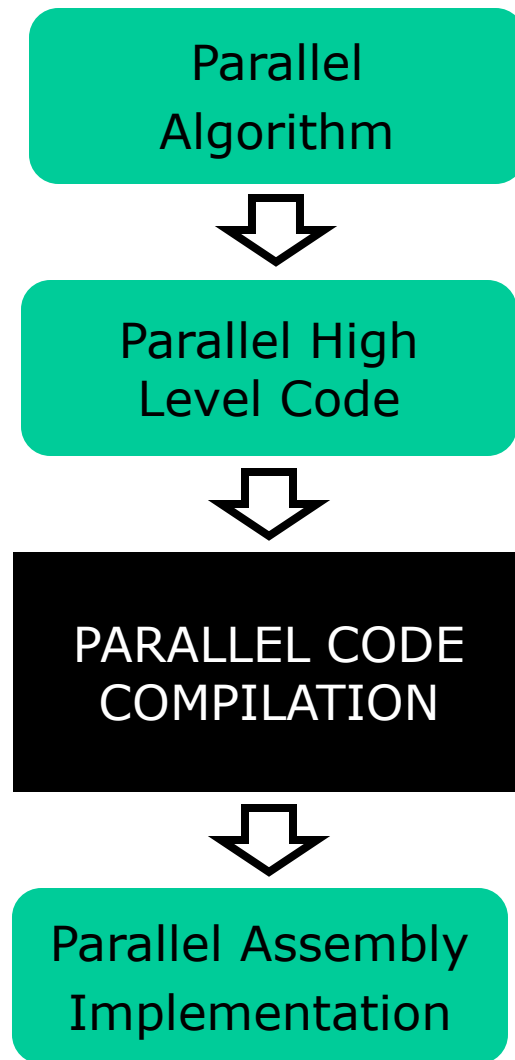
- ❑ `and`, `or`, `b`, `c` can overlap:

```
int array_a[10], array_b[10];
...
and_or(9, &array_a[1], &array_a[1], array_b, array_a);
```

- ❑ The designer maybe will never use the function in this way, but the compiler **does not know...**



- ❑ Complete automatic parallelization is (at the moment?) **not feasible**
- ❑ Tools are not able to extract all the available parallelism from a specification designed to be executed in **sequential way**



- ❑ The programmer needs to give **hints** to the tools
- ❑ Write **parallel algorithms**
- ❑ Implement with high level parallel code
- ❑ Leave only code compilation to the tools

- There are three critical aspects:
 - ▶ Which **type** of parallelism has to be considered
 - ▶ How to **design** the parallel algorithm
 - Trying to parallelize existing sequential algorithms
 - From scratch
 - ▶ How to **provide information** about the parallelism to the tools

Types of Parallelism – 2

Level of Parallelism

9

- ☐ Bits
- ☐ Instructions
- ☐ Tasks

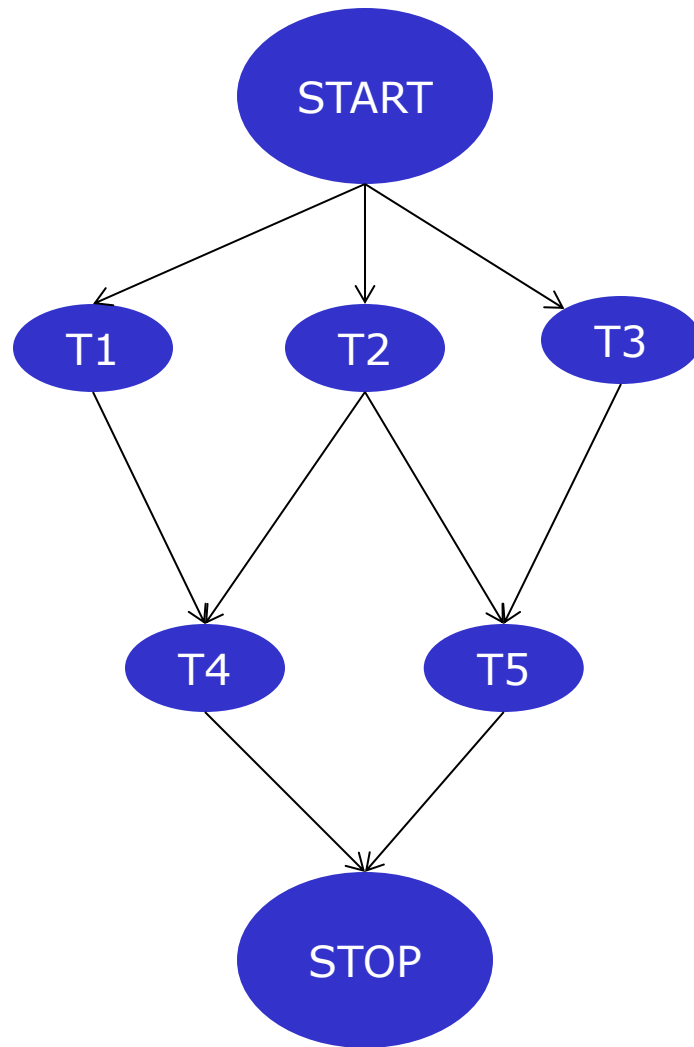
- ❑ Bits composing words represent different data
- ❑ A single instruction can manipulate **different data at a time**
- ❑ It is very relevant in Hardware Implementation of algorithm
- ❑ It can become significant also in software implementation (e.g., representing set of elements as strings of bits)

- ❑ **Different instructions** executed at the **same time** on the same core
- ❑ Supported by multiple execution units, pipeline, vector, SIMD units etc.
- ❑ This type of parallelism can be easily extracted by compilers

- ❑ Task: a logically **discrete section** of computational work
 - ▶ Typically a program or program-like set of instructions that is executed by a processor
- ❑ Parallel program
 - ▶ Multiple tasks running on multiple processors
- ❑ Supported by shared memory, cache coherence mechanisms
- ❑ Usually difficult to be automatically extracted

Task Level Parallelism: Parallel Task Graph

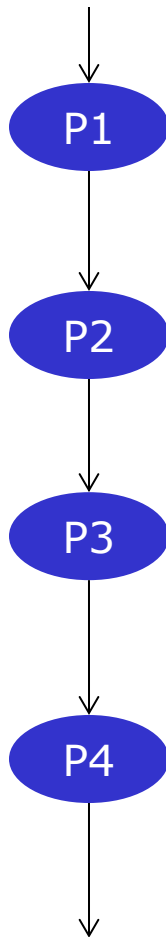
13



- ❑ Vertices correspond to tasks
- ❑ Edges represent precedencies or data communications
- ❑ Each task is executed once (in principle)

Task Level Parallelism: Pipeline

14



- ❑ Like processor pipeline
- ❑ Edges represent data passing
- ❑ Suitable to parallelize streaming elaboration such as audio and video encoding
- ❑ Each task is executed at each stage

- ❑ Two main models for communication
- ❑ **Shared memory:**
 - ▶ All the processors, so all the tasks, share a global memory with the same address space
 - ▶ Modifications in a memory location performed by a processor are seen by all the other processors
- ❑ **Message Passing:**
 - ▶ Each task has its private memory
 - ▶ Tasks communicate by explicitly sending and receiving messages

Example of different levels of parallelism

16

```
void and_or(int SIZE, int * restrict and, int * restrict
or, int * restrict b, int * restrict c){
    int i = 0;
    for(i=0; i<SIZE; i++)
        and[i] = b[i] & c[i];
    for(i=0; i<SIZE; i++)
        or[i] = b[i] | c[i];
}
```

- ❑ If memory locations do not overlap (`restrict`) we can extract parallelism at different levels:
 - ▶ **Task** (inter-loops): the two loops can be run in parallel
 - ▶ **Instruction** (intra-loops): each iteration of the loop can run in parallel
 - ▶ **Bit** (intra-operation): each bit can be computed in parallel

- ❑ Design a «good» parallel algorithm by extracting all the available parallelism is **not enough**
 - ▶ **not** all the extracted parallelism is **exploitable** on a real architecture
- ❑ We need to consider which parallelism is available on the considered architecture
 - ▶ Non suitable parallelism can introduce **overhead**
- ❑ We need to «**describe**» the parallelism to the compilation tools to make it exploitable

- ❑ Translation from pseudo-code to high level source code (e.g., C/C++) is usually quite trivial
- ❑ All the sequential machine can be modeled as a von Neumann Architecture
- ❑ Real processors can differ a lot, but compilers are usually able to manage this gap and optimize the application for a given architecture
- ❑ Example:
 - ▶ Intel 80386 and Intel i9 processors are quite different, but you can use the same source code to create an optimized application for them

- ❑ New programming languages (mainly developed for research activity):
 - + Introduced since parallel programming is a different paradigm
 - **did not have big success**:
 - too immature compilers
 - you need to learn a new language
- ❑ Extensions to existing programming language
 - + can be **easily adopted** by designer
 - + can be **easily integrated** in existing compilers
 - can describe **only some types** of parallelism (e.g., pipeline parallelism is difficult to be described)

List of Parallel Programming Languages (from Wikipedia)

20

- ❑ **Actor model**: Axum, Elixir, Erlang, Janus, Red, SALSA, Scala/Akka, Smalltalk, Akka.NET
- ❑ **Coordination languages**: CnC, Glenda, Linda, coordination language, Millipede
- ❑ **Dataflow programming**: CAL, E, Joule, LabView, Lustre, Preesm, Signal, SISAL, BMDFM
- ❑ **Distributed computing**: Bloom, Hermes, Julia, Limbo, MPD, Oz, Sequoia, SR
- ❑ **Event-driven and hardware description**: Esterel, SystemC, SystemVerilog, Verilog, Verilog-AMS, VHDL
- ❑ **Functional programming**: Clojure, Concurrent ML, Elixir, Erlang, Futhark, Haskell, Id, MultiLisp, SequenceL, Elm
- ❑ **Logic programming**: Parlog, Prolog
- ❑ **Monitor-based**: Concurrent Pascal
- ❑ **Multi-threaded**: C=, Cilk, Cilk Plus, C#, Clojure, Fork, Java, ParaSail, Rust, SequenceL

List of Parallel Programming Languages (from Wikipedia)

21

- ❑ **Object-oriented programming**: μ C++, Ada, C*, C#, C++ AMP, Charm++, D Programming Language, Eiffel SCOOP, Emerald, Java, Join Java, ParaSail, Smalltalk
- ❑ **Partitioned Global Address Space (PGAS)**: Chapel, Coarray Fortran, Fortress, High Performance Fortran, Titanium, Unified Parallel C, X10, ZPL
- ❑ **Message passing**: Ateji, Rust
- ❑ **Communicating Sequential Processing**: JCSP, Alef, Ease, FortranM, Go, JoCaml, Joyce, Limbo, Newsqueak, Occam, Occam- π , PyCSP, SuperPascal, XC
- ❑ **APIs/Frameworks**: Apache Hadoop, Apache Spark, Apache Flink, Apache Beam, CUDA, OpenCL, OpenHMPP, OpenMP

Why so many Parallel Programming Languages?

22

- ❑ Translation from pseudo-code to high level source code (e.g., C/C++) can be an **hard task**
- ❑ Parallel architecture are composed of equivalent von Neumann Machines, but
- ❑ Real architectures differ so much that now compilers are **not able to fill the gap** between an abstract model and real implementation
 - ▶ Optimized applications can not be generated starting from generic parallel code
 - ▶ Code extensions have been specialized for particular types of applications/architectures

TECHNOLOGY	TYPE	YEAR
Verilog/VHDL	Languages	1984/1987
MPI	Library	1994
PThread	Library	1995
OpenMP	C/Fortran Extensions	1997
CUDA	C Extensions	2007
OpenCL	C/C++ Extensions + API	2008
Apache Spark	API	2014

- ❑ Message passing and threads are technologies older than standards that defined them
- ❑ New technologies directly introduced as standard

TECHNOLOGY	AUTHORS	DEVELOPERS
Verilog/VHDL	Phil Moorby – Prabhu Goel / United States Department of Defence	Accellera Systems Initiative / IEEE VHDL Analysis and Standardization Group
MPI	MPI Forum	MPI Forum
PThread	IEEE Technical Committee on Operating System	Austin Group
OpenMP	OpenMP Architecture Review Board	OpenMP Architecture Review Board
CUDA	NVIDIA	NVIDIA
OpenCL	Apple	Khronos Group
Apache Spark	Berkeley	Apache Foundation Databricks

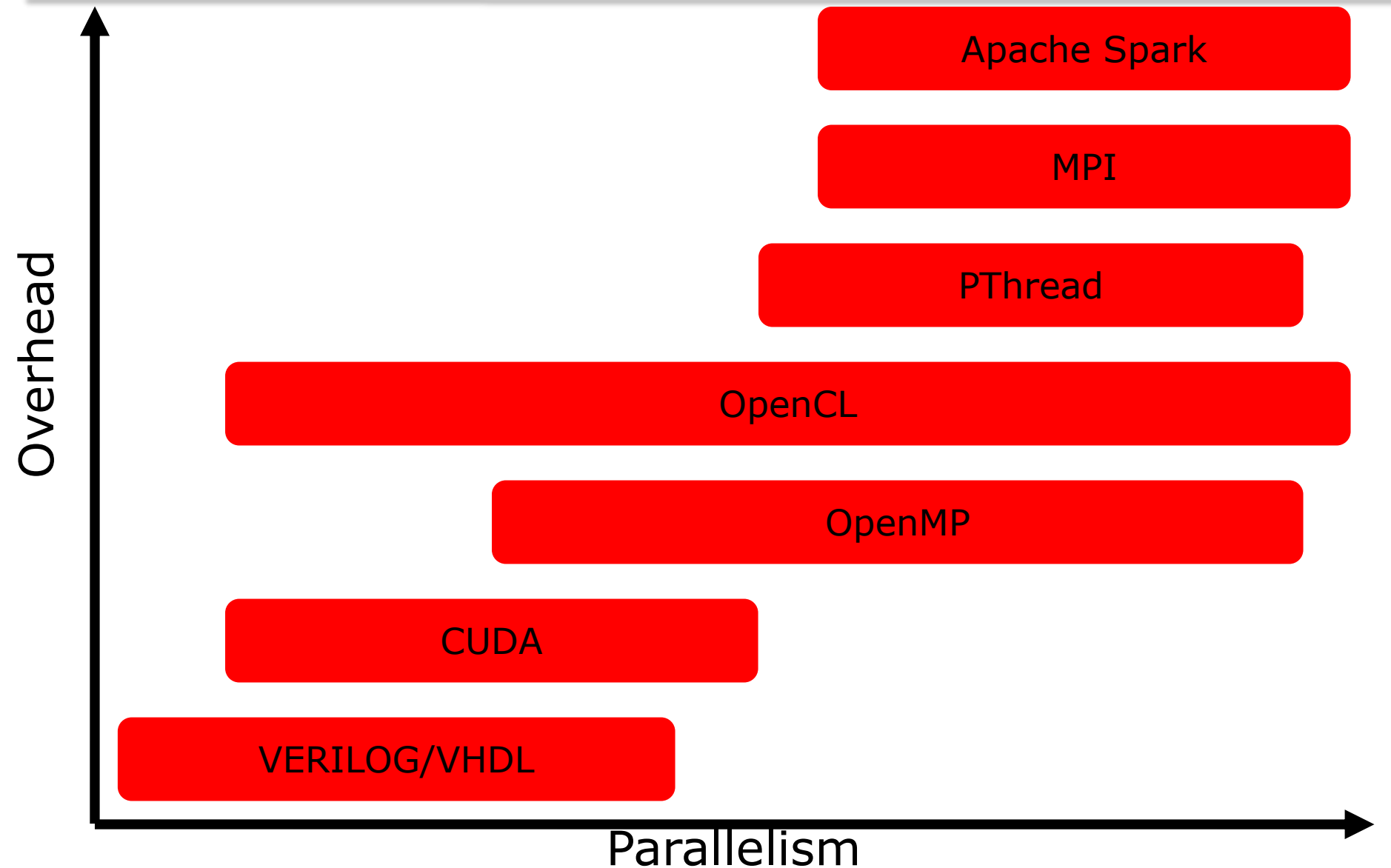
All technologies except CUDA are developed by consortiums/Foundation

TECHNOLOGY	Bit	Instruction	Task
Verilog/VHDL	Yes	Yes	No
MPI	(Yes)	(Yes)	Yes
PThread	(Yes)	(Yes)	Yes
OpenMP	(Yes)	(Yes)	Yes
CUDA	(Yes)	No	(Yes)
OpenCL	(Yes)	No	Yes
Apache Spark	(Yes)	No	(Yes)

- ❑ Bit level parallelism can be exploited on a single core by means of bit-level operators
- ❑ Instruction level parallelism can be exploited through compilers on superscalar cores

Parallelism granularity

26



TECHNOLOGY	Processors	Memory
Verilog/VHDL	ASIC – FPGA	
MPI	Multi CPUs	(Mainly) Distributed Memory
PThread	Multi-core CPU	(Mainly) Shared Memory
OpenMP	Multi-core CPU	(Mainly) Shared Memory
CUDA	CPU + GPU(s)	(Distributed) Shared Memory
OpenCL	Heterogeneous Architecture	Both distributed and shared memory
Apache Spark	Multi CPUs	Distributed Memory

CUDA memory:

- ▶ Distributed between CPU and GPU
- ▶ Shared on the GPU

TECHNOLOGY	Parallelism	Communication
Verilog/VHDL	Explicit	Explicit
MPI	Implicit	Explicit
PThread	Explicit	Implicit
OpenMP	Explicit	Implicit
CUDA	Implicit(Explicit)	Implicit(Explicit)
OpenCL	Explicit/Implicit	Explicit/Implicit
Apache Spark	Implicit	Implicit

❑ CUDA and OpenCL:

- ▶ Different kernels-CPU code: explicit parallelism and communication
- ▶ Threads of the same kernel: implicit parallelism and communication

❑ In OpenCL explicit parallelism can be used also in a single kernel

TECHNOLOGY	Target Independent Code?	Development Platforms
Verilog/VHDL	Yes (behavioral) No (structural)	Mainly Linux
MPI	Yes	All
PThread	Yes	All – Windows through a wrapper
OpenMP	Yes	All – Different compilers
CUDA	Depend on CUDA capabilities	All
OpenCL	Yes	All – Different compilers
Apache Spark	Yes	Mainly Linux

- ❑ Performance optimization usually requires knowledge of the target architecture

TECHNOLOGY	Compilation tool chain	Runtime environment
Verilog/VHDL	Provided by HW vendor	
MPI	Generic compilers / ad-hoc compilers	Support for multi-processes applications
PThread	Generic compilers	Support for multi-threading applications
OpenMP	Compiler with OpenMP support	Depends on OpenMP implementation
CUDA	NVIDIA Toolkit	Based on NVIDIA drivers
OpenCL	Ad-hoc compilers	Depends on the target architecture
Apache Spark	Interpreter/Runtime compilation	Complex runtime system

□ Pros:

- ▶ Complete control on computation and memory
- ▶ No overhead introduced in the computation
- ▶ Provides access to potentially large computational power

□ Cons:

- ▶ Requires specific Hardware (e.g., ASIC or FPGA) to implement functionality
- ▶ Difficult to learn: completely different programming language and programming paradigm
- ▶ Depends on the chosen target architecture

□ Pros:

- ▶ Can be adopted on different types of architectures
- ▶ Scalable solutions
- ▶ Synchronization and data communication are explicitly managed

□ Cons:

- ▶ Communication can introduce significant overhead
- ▶ Programming paradigm more difficult than shared memory based ones
- ▶ Standard does not reflect immediately advances in architecture characteristics

❑ Pros:

- ▶ Can be adopted on different architectures
- ▶ Explicit parallelism and full control over application

❑ Cons:

- ▶ Task management overhead can be significant
- ▶ Not easily scalable solutions
- ▶ Low level API

❑ Pros:

- ▶ Easy to learn
- ▶ Scalable solution
- ▶ Parallel applications can also be executed sequentially

❑ Cons:

- ▶ Mainly focused on shared memory homogeneous systems
- ▶ Require small interaction between tasks

□ Pros:

- ▶ Provides access to the computational power of GPUs
- ▶ Writing a CUDA kernel is quite easy
- ▶ Already optimized libraries

□ Cons:

- ▶ Targets only NVIDIA GPUs
- ▶ Difficult to extract massive parallelism from application
- ▶ Difficult to optimize CUDA kernel

□ Pros:

- ▶ Target-independent standard
- ▶ Hides architecture details
- ▶ Same programming infrastructure for very heterogeneous architecture: CPU + GPU (+FPGA)

□ Cons:

- ▶ Difficult programming paradigm for its heterogeneity
- ▶ Hiding of architecture details makes difficult to obtain best performances

□ Pros:

- ▶ API for different languages
- ▶ Explicit parallelization and communication are not required
- ▶ Preinstalled on cloud provider VMs

□ Cons:

- ▶ Suitable only for big data applications
- ▶ Does not (yet) fully support GPUs

Mix of parallelism technologies:

OpenMP + CUDA

38

- ❑ Allows to exploit Multi-core CPU and GPU
- ❑ CUDA is used to parallelize GPU code
- ❑ OpenMP is used to parallelize CPU code

❑ First scenario:

- ▶ MPI used to express coarser parallelism (Multi CPU)
- ▶ OpenMP used to express finer parallelism (Multi core)

❑ Second scenario:

- ▶ MPI used to implement communications
- ▶ OpenMP used to parallelize computation

Mix of parallelism technologies:

OpenCL + Verilog/VHDL

40

- ❑ In principle, hardware kernels (implemented for example on FPGA) can be used as accelerators
- ❑ OpenCL used to describe parallelism among different processing elements
- ❑ Verilog/VHDL used to describe hardware kernel
- ❑ Example of target: Intel Xeon Scalable (Skylake + Arria 10 FPGA)

- ❑ Separation of the algorithm being implemented from its execution schedule
 - ❑ Execution schedule:
 - ❑ loop nesting, parallelization, loop unrolling and vector instruction.
 - ❑ allows the programmer to experiment with scheduling and finding the most efficient one.
- ❑ Proposed for image processing first by MIT/Stanford/Google
- ❑ Now it is used in deep learning applications

- ❑ Material taken from different sources:
 - ▶ Marco Lattuada material
 - ▶ two tutorials by Blaise Barney from the Lawrence Livermore National Laboratory and from slides of prof. Lanzi and Ing. Loiacono (Algoritmi e Calcolo Parallelo)
 - ▶ Introduction to Parallel Computing
Blaise Barney, Lawrence Livermore National Laboratory
https://computing.llnl.gov/tutorials/parallel_comp/
 - ▶ Also available as Dr.Dobb's "Go Parallel"
Introduction to Parallel Computing: Part 2
Blaise Barney, Lawrence Livermore National Laboratory
 - ▶ "Structured Parallel Programming: Patterns for Efficient Computation," Michael McCool, Arch Robinson, James Reinders, 1st edition, Morgan Kaufmann, ISBN: 978-0-12-415993-8, 2012